Below is a **step-by-step, beginner-friendly but technically correct guide** to using **openpyxl** for **reading and writing Excel (.xlsx) files** in Python.
Each step builds logically, so you can follow it even if this is your **first time working with Excel in Python**.

Step 1: What is openpyxl? (Easy Concept)

- openpyxl is a **Python library** used to:
  - Read Excel files (.xlsx)
  - Write new Excel files
  - Modify existing Excel files
- It works **cell by cell**, just like Excel itself.

Mental model:

Workbook → Worksheet → Cells

Step 2: Install openpyxl

```
pip install openpyxl
```

Verify installation:

```
import openpyxl

print(openpyxl.__version__)
```

Step 3: Create a New Excel File (Writing Basics)

Goal:

Create an Excel file and add data to it.

```
from openpyxl import Workbook


# Create workbook

wb = Workbook()
```

```
# Select active sheet
sheet = wb.active

# Add data
sheet["A1"] = "Name"
sheet["B1"] = "Age"
sheet["C1"] = "Marks"

sheet["A2"] = "Ali"
sheet["B2"] = 20
sheet["C2"] = 85

# Save file
wb.save("students.xlsx")
```

📌 **What happened**

- Workbook() → created Excel file
- sheet["A1"] → accessed a cell
- save() → created .xlsx file

Step 4: Read Excel File (Very Easy)

Goal:

Read data from an existing Excel file.

```
from openpyxl import load_workbook

wb = load_workbook("students.xlsx")
```

```
sheet = wb.active


print(sheet["A1"].value)
print(sheet["A2"].value)
print(sheet["C2"].value)
```

📌 **Key point**

- .value is used to get the actual data from the cell

Step 5: Read All Rows (Important Concept)

```
for row in sheet.iter_rows(values_only=True):
    print(row)
```

Output:

```
('Name', 'Age', 'Marks')
('Ali', 20, 85)
```

✓ values_only=True → gives clean data (no cell objects)

Step 6: Add Data Row by Row (Append Method)

```
sheet.append(["Sara", 21, 90])
sheet.append(["Ahmed", 22, 88])


wb.save("students.xlsx")
```

📌 append() automatically moves to the next row

Step 7: Update Existing Cell (Editing Data)

```
sheet["C2"] = 95   # Change Ali's marks
wb.save("students.xlsx")
```

✓ Excel file updated without deleting old data

Step 8: Read Specific Rows & Columns

Read a column

```
for cell in sheet["A"]:
    print(cell.value)
```

Read a row

```
for cell in sheet[2]:
    print(cell.value)
```

Step 9: Get Total Rows & Columns

```
print(sheet.max_row)
print(sheet.max_column)
```

Useful for loops and automation.

Step 10: Work with Multiple Sheets

Create new sheet

```
sheet2 = wb.create_sheet("Summary")
sheet2["A1"] = "Total Students"
sheet2["B1"] = sheet.max_row - 1
```

Access sheet by name

```
sheet = wb["Summary"]
```

Step 11: Simple Real-World Example

Task:

Calculate average marks and save to Excel.

```
total = 0
count = 0

for row in sheet.iter_rows(min_row=2, min_col=3, max_col=3, values_only=True):
    total += row[0]
    count += 1

average = total / count

summary = wb["Summary"]
summary["A2"] = "Average Marks"
summary["B2"] = average

wb.save("students.xlsx")
```

Step 12: Common Mistakes (Avoid These)

Mistake	Reason	Fix
File not updating	Forgot save()	Always save
Permission error	Excel file open	Close Excel
.xls file	openpyxl supports .xlsx	Convert file
Reading empty cells	No value	Check for None

Step 13: When to Use openpyxl

✓ Excel automation
✓ Small to medium data

✓ Reports & results
❌ Huge datasets (use pandas)

Practice Exercises (Very Important)

1. Create Excel file with 5 students
2. Update one student's marks
3. Read only "Marks" column
4. Add a "Result" sheet
5. Calculate highest marks

Mini Project (Try This)

Student Result Manager

- Input student data
- Save to Excel
- Update marks
- Generate summary sheet

If you want next:

- 📊 Excel **charts with openpyxl**
- 📁 Excel **automation project**
- 🧠 Excel **interview questions**
- 🔄 Compare **openpyxl vs pandas**

Just tell me what you want next.